CS 474 Final Project

Learning Control Barrier Functions and Controller for Navigation

James Wade

I. Introduction

Control Barrier Functions (CBFs) provide formal safety guarantees for autonomous systems by defining safe regions (**invariant sets**) of the state space. Traditionally, constructing CBFs and computing safe control inputs requires analytical solutions specific to each obstacle geometry, which becomes cumbersome for complex or heterogeneous environments. Recent advances in machine learning suggest an alternative: training neural networks to approximate CBF-based controllers [1], [2]. My research in the engineering department focuses on this topic, and this project investigates whether neural networks can learn valid CBFs for 2D navigation among convex obstacles, with performance comparable to analytical baselines.

A. Background

The invariant set, S, is defined by a **barrier function** $h(x)$, where x is the agent's state. The function is positive when the agent is within the invariant set and negative otherwise; that is, for safety:

$$h(x) \geq 0 \quad \forall x \in S \tag{1}$$

To ensure invariance, the derivative $\dot{h}(x)$ must satisfy:

$$\dot{h}(x) \geq -\alpha h(x) \quad \forall x \in S \tag{2}$$

Equation 2 ensures that as $h(x)$ approaches zero, $\dot{h}(x) \geq 0$, keeping the agent within S. Analytical computation of $h(x)$ and $\dot{h}(x)$ is feasible for simple convex obstacles, but challenging for non-convex shapes. Neural networks can potentially learn $h(x)$ and $\dot{h}(x)$ directly (Neural CBFs, NCBF).

II. Dataset Composition and Environment

$h_i(x)$ and its gradient depend on the robot position and obstacle locations and radii. In multi-obstacle environments, the barrier with the smallest value (i.e., the closest obstacle) dominates safety. Table I shows the layout of the dataset. This dataset was generated with uniformly random test positions, obstacle locations, and obstacle radii. The analytical barrier function and gradient were then calculated at each test point.

TABLE I: Layout of the CBF dataset

Input	Description
x	Robot's x-position in the environment
y	Robot's y-position in the environment
$R_{\min}$	Radius of the closest obstacle
$o_{\min,x}$	x-coordinate of closest obstacle center
$o_{\min,y}$	y-coordinate of closest obstacle center
d_{min}	Robot's distance to the closest obstacle edge
Output	**Description**
h_{true}	True barrier function value for the closest obstacle
∇h_x	x-component of barrier function gradient for closest obstacle
∇h_y	y-component of barrier function gradient for closest obstacle

The environment consists of a single agent and multiple circular obstacles of various radii. The agent can be commanded to move holonomically in the x and y directions independently, allowing unconstrained kinematic motion.

TABLE II: Layout of the RL observations

Item	Description
x	Robot's x-position in the environment
y	Robot's y-position in the environment
g_x	x-coordinate of goal position
g_y	y-coordinate of goal position
$\Delta g_{\min,x}$	Relative x-distance from robot to goal
$\Delta g_{\min,y}$	Relative y-distance from robot to goal
d_{min}	Robot's distance to the closest obstacle edge

Additionally, as the project progressed, I decided to train a PPO-based controller that progresses the robot towards the goal. Table II shows the structure of the RL states. The action space had two dimensions (velocity in x and velocity in y) and was bounded from -1 to 1 by a tanh function. A vectorized gymnasium environment was created to support data generation.

III. CBF Network Methods and Results

Figure 3 shows the comparison of the analytical CBF, an NCBF learned with a vanilla NN, and an NCBF learned with a ResNet architecture. Trajectory performance of these CBFs is shown with both a straight-line nominal controller (in which the applied velocity is in the direction of the goal) and a PPO-based controller. A 70/30 train-test split was used. The loss function used to train the CBF was a summation of the MSE for the predicted barrier value h and the MSE for the predicted barrier gradient ∇h. The Adam optimizer was used with a learning rate of 0.001, and weights were initialized with He initialization. I experimented with what features I should input into the NCBF, and I eventually settled on those listed in Table I.

A. Fully-Connected Neural Network

The first method I tried was a fully-connected neural network. I found that I did not need to train the network for very long (10-20 epochs through the dataset) before the validation score began to rise. The final model had two hidden layers with 128 hidden units each. The loss decreased monotonically with this structure. However, I noticed that the fully-connected network quickly devolved if it were any deeper. The training became much noisier, supporting the fact that deep vanilla NNs suffer from extreme gradient changes.

B. Residual Neural Network

To address the training instability of deep vanilla NNs, I implemented a ResNet architecture for the NCBF. The final model had six residual blocks, each with a width of 128 units. Compared to the vanilla NN, the loss decreased faster and reached a lower value for the same number of epochs (Figure 1). However, the training was not as monotonic as the vanilla NN.

C. CBF Comparison

Figure 3 shows that both NCBFs kept the robot from colliding with any obstacles as it progressed towards its goal. Figure 4 shows the barrier function value $h(x)$ as a

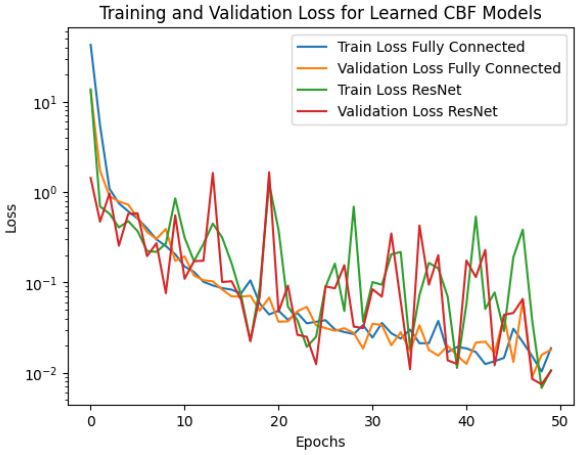

Fig. 1: Loss function for the CBF training

heat map across the environment. The gray circles represent the boundaries of the circular obstacles (compare with the obstacles in Figure 3), and the dotted black lines represent the level curve where $h(x) = 0$. The optimal performance of the NCBFs would have the dotted lines directly on top of the gray circles, as shown in the analytical CBF heatmap. Both NCBFs estimated the barrier function quite well, with the fully-connected NCBF performing better than the ResNet CBF. Both performed equally poorly on the smallest obstacle in the top-right corner of the environment. I believe, however, that the ResNet CBF is more flexible to non-convex obstacles and would be a better option given more time to explore that facet of the application.

IV. Controller Methods and Results

A. Nominal Straight-Line Controller

This controller served as the baseline method. The agent's commanded velocity is along the line extending from the agent's current position to the goal location. The magnitude of the velocity was always unity.

B. PPO-based Controller

Using PPO and GAE for the advantage estimation, two RL controllers were trained — one without obstacle avoidance and one with obstacle avoidance. The agent was rewarded incrementally for making progress toward the goal and was rewarded for reaching the goal. The agent was punished incrementally within a certain range of each obstacle as it got closer. However, the RL trained with obstacle avoidance was never successful, so Figure 3 shows the combined result of an obstacle-ignorant RL controller with a CBF safety filter. Similar to the CBF models, I experimented with using ResNets as the actor-critic models, but the controller exhibited better control with a vanilla NN structure for the actor-critic. Both the critic and actor models had eight hidden layers, each with 128 hidden units. The actor has a final tanh activation function in order to broadcast the actions between -1 and 1. The Adam optimizer was used with a learning rate of 0.001, weights were initialized with He initialization, and the standard PPO loss functions were used. The training

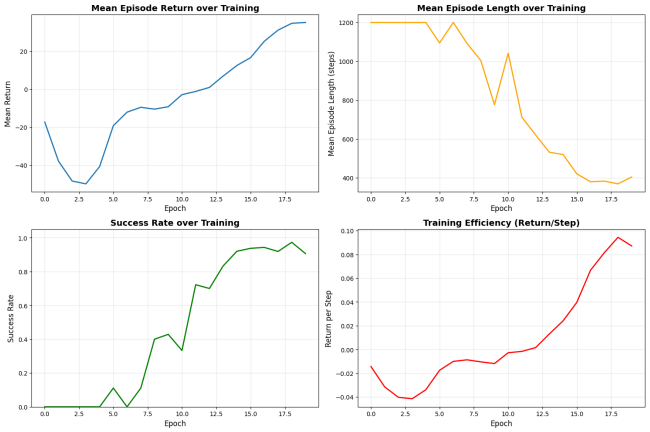

Fig. 2: PPO training results for the obstacle-unaware model

results are shown in Figure 2.

C. Controller Comparison

Figure 5 shows the resultant vector fields for a goal position in the upper half of the environment. The optimal performance would be that all vectors point directly towards the goal, as shown with the nominal straight-line controller. The obstacle-unaware RL controller approximates this quite well, whereas the obstacle-avoidant RL controller never found a good solution. A possible reason for this was saturation in the tanh function. I suspect that more tuning of the hyperparameters was needed due to the non-convexity of the environment's safe regions.

The RL model can be improved in its reward structure. Ignoring obstacles, I am only rewarding the agent for making progress toward the goal location. However, no reward was provided for approach speed, so the agent was not explicitly incentivized to take direct paths. Despite this, Figure 2 does show that the average rollout length decreased with each epoch, and that the return per step ratio increased during training. Enhancing the reward function is the next research step.

V. Conclusions and Learning Outcomes

I learned a lot from this project. I learned 1) how to create my own vectorized gymnasium environment, 2) how to use GAE for advantage estimation in PPO, as well as feeling more confident in PPO, 3) how to think critically about input features, 4) how to formulate my own loss formulas for training, and 5) how to apply deep learning principles to classic control problems. Overall, this has been a very rewarding project on the application of deep learning to robotic control.

References

[1] M. Harms, M. Kulkarni, N. Khedekar, M. Jacquet, and K. Alexis, "Neural control barrier functions for safe navigation," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 10 415–10 422.

[2] M. Yu, C. Yu, M.-M. Naddaf-Sh, D. Upadhyay, S. Gao, and C. Fan, "Efficient motion planning for manipulators with control barrier function-induced neural controller," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 348–14 355.

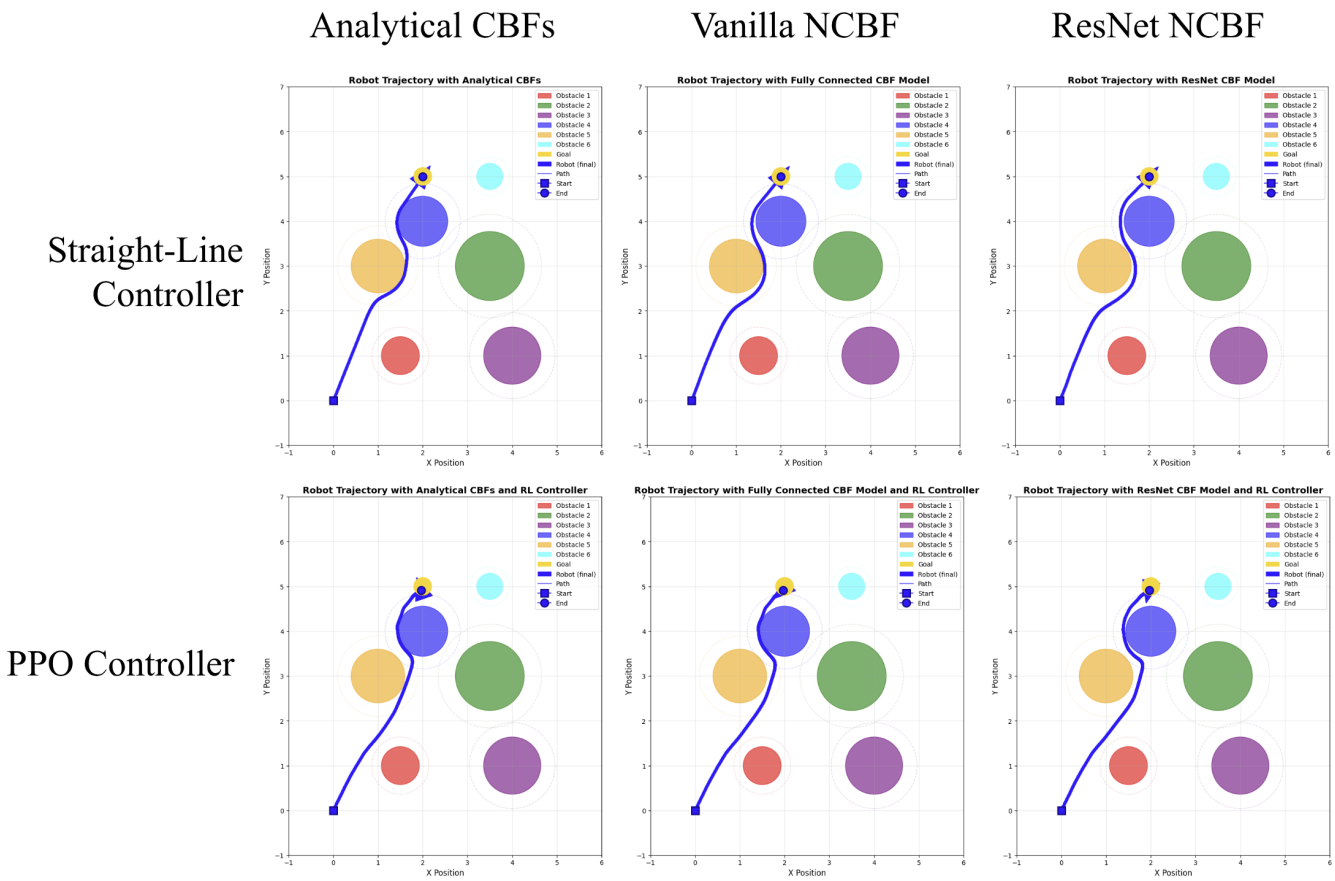

Fig. 3: Combined figure comparing the analytical CBFs, an NCBF with a vanilla neural network, and an NCBF using a ResNet architecture. These are each compared using a straight-line analytical controller and a PPO-based controller.

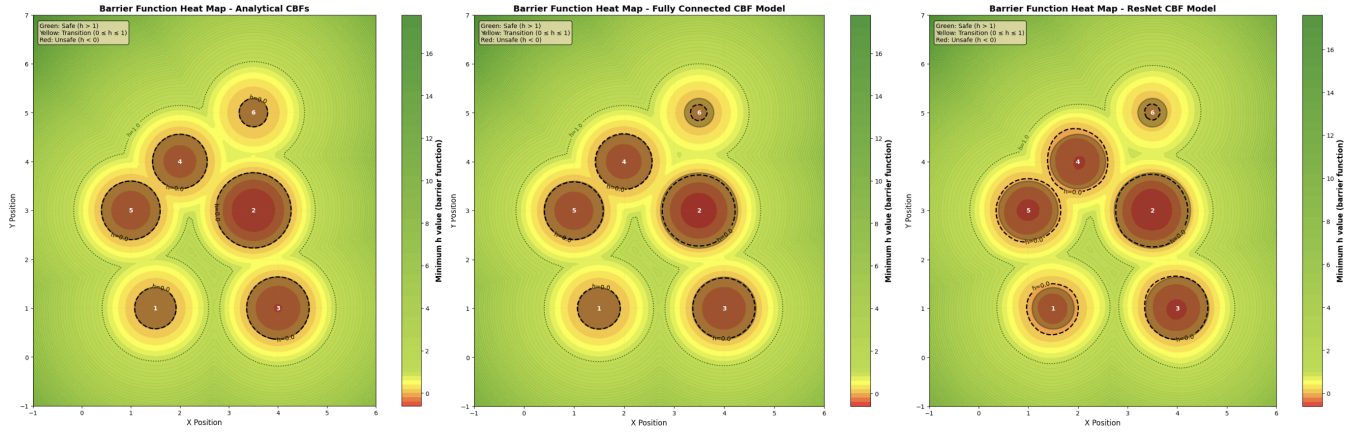

Fig. 4: Heatmaps showing the barrier function value $h(x)$ for the closest obstacle. The gray circle represents the outer boundary of the obstacles, and the black dotted lines are where $h(x) = 0$. Optimal performance is when the black dotted line lies on top of the obstacle boundaries.

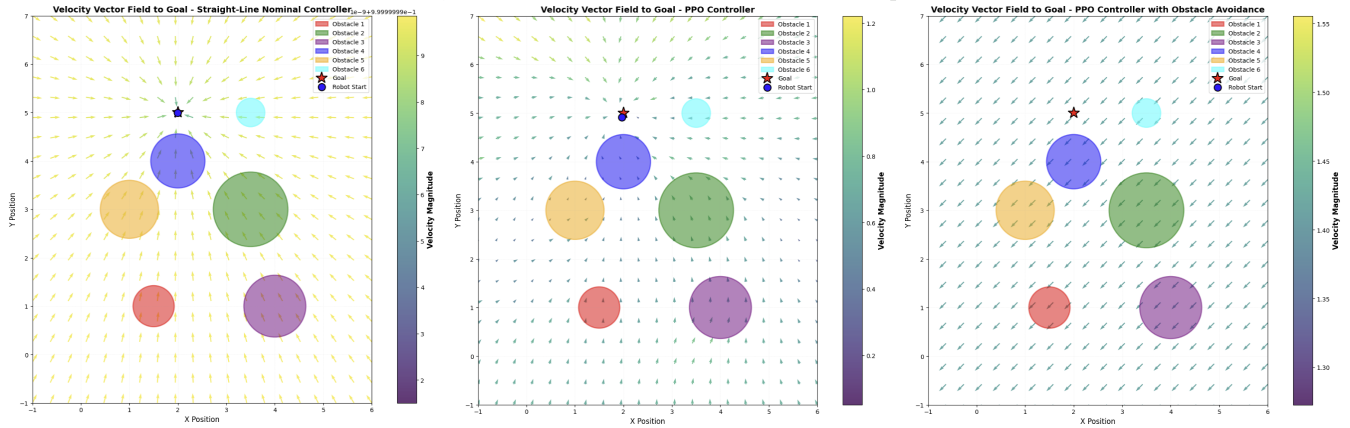

Fig. 5: Vector fields for the analytical straight-line controller, PPO-based controller without obstacle avoidance, and PPO-based controller with obstacle avoidance